

Documentação Completa do JSBase 3.2

1. Introdução

O JSBase é uma linguagem de programação simplificada, criada com o objetivo de facilitar o aprendizado de lógica de programação para estudantes. Inspirado na sintaxe do JavaScript, ele abstrai complexidades, permitindo que os alunos se concentrem nos conceitos fundamentais. A versão 3.2 aprimora ainda mais a robustez e a usabilidade, corrigindo o tratamento de valores booleanos e otimizando o processamento de comandos multilinhas.

Desde sua concepção, o JSBase tem evoluído para oferecer uma experiência de aprendizado intuitiva e eficaz, tornando-se uma ferramenta valiosa para o ensino de programação no SENAI.

2. Funcionalidades e Novidades da Versão 3.2

A versão 3.2 do JSBase integra todas as funcionalidades desenvolvidas até o momento, com um foco especial na correção de bugs e na melhoria da experiência de escrita de código. As principais características e inovações incluem:

- **✓ Tipos de Variáveis:** Suporte a `numero`, `texto` e `booleano`.
- **✓ Atribuição Flexível:** Comando `recebe` para atribuição de valores, incluindo suporte a expressões matemáticas complexas e atribuições multilinhas aprimoradas.
- **✓ Saída de Dados:** Comando `mostrar` com suporte a múltiplos argumentos, exibindo informações no console de saída.
- **✓ Entrada de Dados Interativa:** O comando `perguntar` exibe a mensagem e um campo de digitação diretamente no console de saída, aguardando a interação do usuário de forma assíncrona.
- **✓ Condicionais:** Estruturas `se`, `senão` e `fimse` para controle de fluxo.

- **✓ Laços de Repetição:** `enquanto` (equivalente a `while`) e `para` (equivalente a `for`) para execução repetitiva de blocos de código.
- **✓ Validação Estrutural de Blocos:** Verificação rigorosa de que todos os blocos de código (`inicio / fim`, `se / fimse`, `enquanto / fimenquanto`, `para / fimpara`) são corretamente abertos e fechados, com mensagens de erro precisas.
- **✓ Sistema de Erros Aprimorado:** Mensagens de erro claras com indicação da linha exata onde o problema ocorreu, facilitando a depuração.
- **✓ Case Insensitive:** A linguagem é insensível a maiúsculas e minúsculas para todas as palavras-chave, proporcionando maior flexibilidade na escrita do código.
- **✓ Robustez no Processamento:** Expressões regulares (Regex) otimizadas para maior flexibilidade de espaçamento e posicionamento de comandos, incluindo a capacidade de unir comandos `recebe` que se estendem por múltiplas linhas.
- **✓ Editor Aprimorado:** Sincronização de scroll perfeita entre o `textarea` e o `highlight`, e destaque de sintaxe para palavras reservadas, strings, números e comentários.
- **✓ Execução Assíncrona:** O motor do JSBase agora utiliza `async/await` para gerenciar operações que exigem espera, como a entrada de dados, sem bloquear a interface do usuário.
- **✓ Funções Nativas de Conversão de Tipo:** Introdução das funções `numero()`, `texto()` e `booleano()` para converter valores entre tipos.
- **✓ Suporte a Valores Booleanos Literais: NOVIDADE!** As palavras `verdadeiro` e `falso` agora são reconhecidas nativamente como valores booleanos, eliminando o erro “is not defined”.

3. Sintaxe da Linguagem JSBase 3.2

Esta seção detalha a sintaxe de todos os comandos e estruturas disponíveis no JSBase 3.2, juntamente com seus equivalentes em JavaScript. As palavras-chave são **case insensitive**.

3.1. Blocos Globais (`inicio` / `fim`)

Delimitam o escopo principal do programa. São opcionais, mas recomendados para organização e validação estrutural.

Comando JSBase	Equivalente JavaScript	Descrição
<code>inicio</code>	<code>{</code>	Marca o início do bloco principal do programa.
<code>fim</code>	<code>}</code>	Marca o fim do bloco principal do programa.

Exemplo:

```
inicio
    mostrar "Meu primeiro programa JSBase!"
fim
```

3.2. Declaração e Atribuição de Variáveis (`recebe`)

O JSBase suporta `numero`, `texto` e `booleano`. A atribuição de valores é feita com `recebe`, suportando expressões e valores multilinhas.

Comando JSBase	Equivalente JavaScript	Descrição
<code>numero minhaIdade recebe</code> <code>25</code>	<code>let minhaIdade = 25;</code>	Declara uma variável numérica e atribui um valor.
<code>texto meuNome recebe</code> <code>"João"</code>	<code>let meuNome =</code> <code>"João";</code>	Declara uma variável de texto e atribui um valor.
<code>booleano estaAtivo recebe</code> <code>verdadeiro</code>	<code>let estaAtivo =</code> <code>true;</code>	Declara uma variável booleana e atribui um valor.
<code>minhaIdade recebe</code> <code>minhaIdade + 1</code>	<code>minhaIdade =</code> <code>minhaIdade + 1;</code>	Atribui um novo valor a uma variável existente.

Exemplo com atribuição multilinhas e expressão:

```
NUMERO idade recebe 10
texto nome recebe "Maria"
booleano ativo recebe verdadeiro // Agora reconhece 'verdadeiro'

idade RECEBE (idade + 5) * 2
mostrar idade

texto mensagem recebe
"Esta é uma mensagem longa."
mostrar mensagem
```

3.3. Saída de Dados (mostrar)

Exibe informações no console de saída.

Comando JSBase	Equivalente JavaScript	Descrição
<code>mostrar "Olá, mundo!"</code>	<code>console.log("Olá, mundo!");</code>	Exibe uma string literal.
<code>mostrar minhaVariavel</code>	<code>console.log(minhaVariavel);</code>	Exibe o valor de uma variável.
<code>mostrar "Nome:", meuNome</code>	<code>console.log("Nome:", meuNome);</code>	Exibe múltiplos valores separados por vírgula.

Exemplo:

```
texto saudacao recebe "Bem-vindo ao JSBase!"
mostrar saudacao
mostrar "O resultado é:", 10 + 20
```

3.4. Entrada de Dados (perguntar)

Solicita uma entrada de texto ao usuário diretamente no console de saída. A execução do programa é pausada até que o usuário digite e pressione Enter.

Comando JSBase	Equivalente JavaScript	Descrição
texto nomeUsuario recebe perguntar "Qual seu nome?"	let nomeUsuario = await lerDoConsole("Qual seu nome?");	Solicita um texto ao usuário e armazena na variável.

Exemplo:

```
texto nome recebe perguntar "Digite seu nome:"
mostrar "Olá,", nome
```

3.5. Funções Nativas de Conversão de Tipo (numero() , texto() , booleano())

Essas funções são essenciais para manipular dados obtidos de `perguntar`, que sempre retorna uma string. Elas permitem converter valores entre os tipos básicos do JSBase.

Função JSBase	Equivalente JavaScript	Descrição
numero(valor)	Number(valor)	Converte o <code>valor</code> para o tipo <code>numero</code> . Útil para cálculos matemáticos.
texto(valor)	String(valor)	Converte o <code>valor</code> para o tipo <code>texto</code> .
booleano(valor)	Boolean(valor)	Converte o <code>valor</code> para o tipo <code>booleano</code> .

Exemplo:

```
numero idadeTexto recebe perguntar "Qual sua idade?"
numero idadeNumerica recebe numero(idadeTexto)
mostrar "Sua idade em número é:", idadeNumerica + 5
```

3.6. Condicionais (se , senão , fimse)

Permitem a execução condicional de blocos de código.

Comando JSBase	Equivalente JavaScript	Descrição
<code>se (idade maior que 18)</code>	<code>if (idade > 18) {</code>	Inicia um bloco condicional <code>if</code> .
<code>senão</code>	<code>} else {</code>	Inicia um bloco <code>else</code> (opcional).
<code>fimse</code>	<code>}</code>	Encerra o bloco condicional.

Exemplo:

```

numero idade recebe 20
se (idade maior que 18)
    mostrar "Maior de idade"
senão
    mostrar "Menor de idade"
fimse

```

3.7. Laços de Repetição (`enquanto` , `para` , `fimenquanto` , `fimpara`)

Executam um bloco de código repetidamente.

Comando JSBase	Equivalente JavaScript	Descrição
<code>enquanto (contador menor que 5)</code>	<code>while (contador < 5)</code> <code>{</code>	Inicia um laço de repetição <code>while</code> .
<code>fimenquanto</code>	<code>}</code>	Encerra o laço <code>enquanto</code> .
<code>para (i recebe 0; i menor que 5; i recebe i + 1)</code>	<code>for (let i = 0; i < 5; i++) {</code>	Inicia um laço de repetição <code>for</code> .
<code>fimpara</code>	<code>}</code>	Encerra o laço <code>para</code> .

Exemplo `enquanto` :

```

numero contador recebe 0
enquanto (contador menor que 3)
    mostrar "Contador:", contador
    contador recebe contador + 1
fimenquanto

```

Exemplo para :

```
para (numero i recebe 0; i menor que 5; i recebe i + 1)
  mostrar "Iteração:", i
fimpara
```

3.8. Comentários (//)

Comentários são linhas de texto ignoradas pelo interpretador, usadas para documentar o código.

Comando JSBase	Equivalente JavaScript	Descrição
// Este é um comentário	// Este é um comentário	Comentário de linha única.

Exemplo:

```
// Este programa calcula a soma de dois números
numero a recebe 10
numero b recebe 20
mostrar a + b // Exibe o resultado
```

3.9. Operadores

O JSBase suporta operadores matemáticos, relacionais e lógicos, que são traduzidos para seus equivalentes em JavaScript.

3.9.1. Operadores Relacionais

Operador JSBase	Equivalente JavaScript	Descrição
maior ou igual	<code>>=</code>	Maior ou igual a.
menor ou igual	<code><=</code>	Menor ou igual a.
maior que	<code>></code>	Maior que.
menor que	<code><</code>	Menor que.
diferente	<code>!=</code>	Diferente de.
igual	<code>==</code>	Igual a.

3.9.2. Operadores Lógicos

Operador JSBase	Equivalente JavaScript	Descrição
e	<code>&&</code>	Operador lógico AND.
ou	<code> </code>	Operador lógico OR.

Exemplo:

```
numero x recebe 10
numero y recebe 5

se (x maior que y e y diferente 0)
    mostrar "Condição verdadeira"
fimse
```

4. Arquitetura Técnica

O JSBase 3.2 opera como um transpiler cliente-side, convertendo o código JSBase para JavaScript diretamente no navegador. A arquitetura é composta por:

- **HTML:** Estrutura da página, incluindo o editor (`textarea`), a área de destaque (`div` com `pre`) e a área de saída (`div#saida`).

- **CSS:** Estilização do editor, destaque de sintaxe e layout geral, incluindo estilos para o input interativo no console.
- **JavaScript (Core):**
 - **JSBase_Nativo** : Um objeto que encapsula as funções nativas do JSBase (numero , texto , booleano). Essas funções são injetadas no escopo de execução do eval() para que o código do aluno possa utilizá-las.
 - **lerDoConsole(mensagem)** : Uma função assíncrona que cria um elemento input dentro da saidaDiv , exibe a mensagem , foca no input e retorna uma Promise que é resolvida com o valor digitado pelo usuário ao pressionar Enter. Isso permite a interação direta no console.
 - **transpilar(codigoJSBase)** : A função central que processa o código. Ela realiza um pré-processamento aprimorado para unir linhas de comandos recebe e perguntar que foram quebradas. Em seguida, percorre o código linha por linha, aplicando expressões regulares para converter comandos JSBase (variáveis, mostrar , se , enquanto , para , perguntar) em seus equivalentes JavaScript. Gerencia a pilhaBlocos para a validação estrutural de todos os blocos.
 - **transpilarLinhaSimples(linha)** : Função auxiliar para transpilar partes de comandos como as condições do para .
 - **traduzirOperadores(texto)** : Converte operadores relacionais e lógicos do português para símbolos JavaScript, e agora também verdadeiro para true e falso para false .
 - **executar()** : Uma função async . Ela limpa o console, captura o código do editor, chama transpilar() , e então executa o JavaScript resultante usando eval() dentro de uma função assíncrona auto-executável (async () => { ... })(). As funções nativas (numero , texto , booleano) são explicitamente injetadas no escopo de eval . Intercepta console.log para exibir a saída na saidaDiv e implementa o tratamento de erros aprimorado, incluindo a detecção de erros de validação de blocos e erros de execução.
 - **updateHighlight()** : Responsável pelo destaque de sintaxe em tempo real, aplicando classes CSS a palavras reservadas, strings, números, comentários e agora também perguntar , numero , texto , booleano , verdadeiro , falso .

- **Sincronização de Scroll:** Event listeners garantem que o scroll do `textarea` e do `highlight` estejam sempre sincronizados.

5. Considerações de Segurança

O JSBase 3.2 utiliza a função `eval()` do JavaScript para executar o código transpilado. Embora seja uma ferramenta poderosa para o propósito educacional de um transpiler cliente-side, é crucial entender que o uso de `eval()` em ambientes de produção com código de fontes não confiáveis é **fortemente desencorajado** devido a riscos de segurança, como a injeção de código malicioso. Para o contexto controlado do JSBase, onde o foco é o aprendizado, esse risco é aceitável. Para aplicações mais complexas, alternativas como a construção de uma Abstract Syntax Tree (AST) e um interpretador seguro seriam preferíveis.

6. Como Usar

1. Abra o arquivo `jsbase_v3.2.html` em qualquer navegador web moderno.
2. Digite seu código JSBase na área de texto principal.
3. Clique no botão “Executar Código” para ver o resultado na área de saída abaixo.
4. Para comandos `perguntar`, digite sua resposta no campo que aparecerá no console e pressione Enter.
5. Observe o destaque de sintaxe em tempo real enquanto digita.
6. Em caso de erros, a mensagem de erro indicará a linha aproximada onde o problema ocorreu, incluindo erros de validação estrutural de blocos.

7. Exemplos de Código

Exemplo 1: Programa Interativo Completo com Conversão de Tipo

```
inicio
    mostrar "--- CALCULADORA DE MÉDIA INTERATIVA ---"

    texto nomeAluno recebe perguntar "Digite o nome do aluno:"
    texto nota1Texto recebe perguntar "Digite a primeira nota:"
    texto nota2Texto recebe perguntar "Digite a segunda nota:"

    // Converte as notas de texto para número usando a função nativa numero()
    numero nota1 recebe numero(nota1Texto)
    numero nota2 recebe numero(nota2Texto)

    numero media recebe (nota1 + nota2) / 2

    mostrar "\nAluno:", nomeAluno
    mostrar "Média Final:", media

    se (media maior ou igual 7)
        mostrar "STATUS: APROVADO!"
    senão
        mostrar "STATUS: REPROVADO!"
    fimse
fim
```

Exemplo 2: Laço para com Interação e Conversão

```
inicio
    texto limiteTexto recebe perguntar "Quantas vezes você quer que eu repita?"
    numero limite recebe numero(limiteTexto)

    para (numero i recebe 1; i menor ou igual limite; i recebe i + 1)
        mostrar "Repetição número:", i
    fimpara
    mostrar "Contagem finalizada."
fim
```

Exemplo 3: Condicional e Variáveis com Conversão para Texto e Booleano

```
inicio
    numero valorNumerico recebe 123
    texto valorEmTexto recebe texto(valorNumerico)
    mostrar "O valor numérico é:", valorNumerico
    mostrar "O valor em texto é:", valorEmTexto

    booleano estaLogado recebe verdadeiro
    se (estaLogado igual verdadeiro)
        mostrar "Usuário está logado."
    senão
        mostrar "Usuário não está logado."
    fimse

    texto frutaFavorita recebe perguntar "Qual sua fruta favorita?"

    se (frutaFavorita igual "maçã" ou frutaFavorita igual "Maçã")
        mostrar "Excelente escolha! Maças são deliciosas."
    senão
        mostrar frutaFavorita, "também é uma boa fruta."
    fimse
fim
```
